

Rapport Technique

BigYellowData

*Pipeline de data engineering pour l'analyse
des taxis jaunes de New York*

Nadir NEHILI • Nahil EL BEZZARI • Yassine LAZIZI •
Pascal LE • Anis MELAIMI

Technologies utilisées :

Apache Spark • PostgreSQL • MinIO
Streamlit • scikit-learn • Airflow
Docker • Python • Scala

Version 1.0
Février 2026

Table des matières

1	Introduction	2
1.1	Contexte du Projet	2
1.2	Objectifs	2
1.3	Stack Technologique	2
2	Architecture Technique	2
2.1	Vue d'Ensemble	2
2.2	Flux de Données	3
2.3	Schéma de Réseau	3
3	Infrastructure et Services	3
3.1	Cluster Apache Spark	3
3.1.1	Spark Master	3
3.1.2	Spark Workers (x2)	3
3.2	MinIO (Data Lake)	3
3.3	PostgreSQL (Data Warehouse)	3
4	Récupération des Données	4
5	Ingestion et Nettoyage	4
5.1	Analyse des données	4
5.2	Nettoyage des données	5
6	Création du Data Warehouse	6
6.1	Choix du schéma	6
6.2	Explications	10
7	Dashboard Analytique	11
8	Service de prédiction ML	14
8.1	Modèle	14
8.2	Entrainement et évaluation	14
8.3	Architecture Modulaire	15
9	Orchestration Airflow	16
9.1	DAG 1 : Pipeline Complet (Manuel)	16
9.2	DAG 2 : Rafraîchissement Mensuel Automatique	16
9.2.1	Fonctionnement du Téléchargement Automatique	17
10	Conclusion	18
11	Annexe	18

1 Introduction

1.1 Contexte du Projet

Le projet **BigYellowData** est une solution complète de data engineering qui implémente un pipeline ETL (Extract, Transform, Load) de bout en bout pour l'analyse des données des taxis jaunes de New York City. Ces données, fournies par la NYC Taxi and Limousine Commission (TLC), représentent des millions de courses de taxi avec des informations détaillées sur les trajets, les tarifs et les modes de paiement.

1.2 Objectifs

Les objectifs principaux du projet sont :

- Construire un **Data Lake** pour le stockage des données brutes
- Implémenter un pipeline de **nettoyage et transformation** des données
- Concevoir un **Data Warehouse** optimisé pour l'analyse OLAP
- Développer un **dashboard interactif** pour la visualisation
- Créer un **service de prédiction ML** pour estimer les prix des courses
- Mettre en place une **orchestration automatisée** du pipeline

1.3 Stack Technologique

TABLE 1 – Technologies utilisées par couche

Couche	Technologies
Traitement Big Data	Apache Spark 3.5.5, Scala 2.13
Data Lake	MinIO (compatible S3), format Parquet
Data Warehouse	PostgreSQL 15
Visualisation	Streamlit, Plotly
Machine Learning	scikit-learn, Random Forest
Orchestration	Apache Airflow
Conteneurisation	Docker, Docker Compose

2 Architecture Technique

2.1 Vue d'Ensemble

L'architecture du projet suit un modèle de pipeline de données moderne avec séparation claire des responsabilités entre les différentes couches.

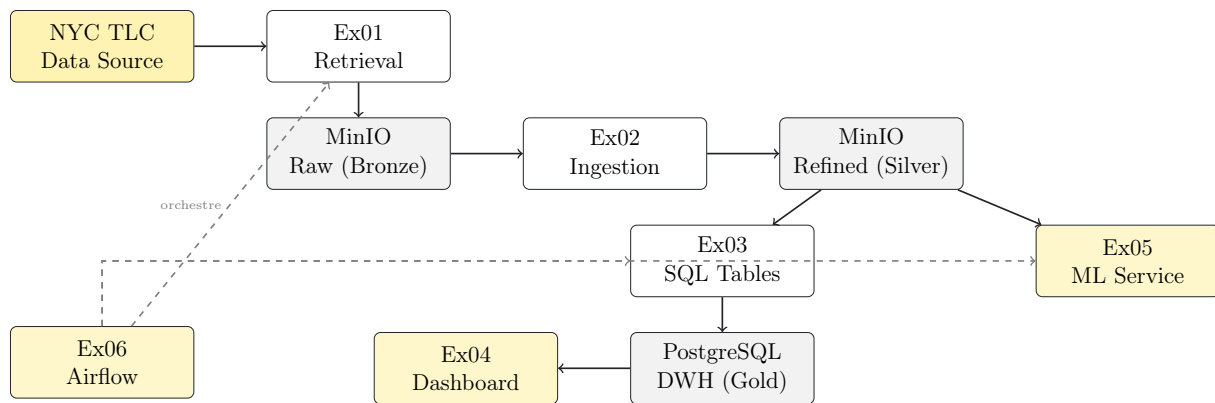


FIGURE 1 – Architecture globale du projet BigYellowData

2.2 Flux de Données

Le flux de données suit une architecture Medallion en trois couches :

1. **Couche Bronze (Raw)** – MinIO : Données brutes.
2. **Couche Silver (Refined)** – MinIO : Données nettoyées et partitionnées.
3. **Couche Gold (Aggregated)** – PostgreSQL : Tables optimisées pour l'analyse.

2.3 Schéma de Réseau

Tous les services communiquent via un réseau Docker bridge nommé `spark-network`.

3 Infrastructure et Services

3.1 Cluster Apache Spark

Le cluster Spark est composé de 3 conteneurs :

3.1.1 Spark Master

- **Rôle** : Coordinateur du cluster
- **Ports** : 7077 (RPC), 8081 (Web UI)

3.1.2 Spark Workers (x2)

- **Rôle** : Exécution des tâches
- **Mémoire** : 2 GB chacun

3.2 MinIO (Data Lake)

MinIO fournit un stockage objet compatible S3.

3.3 PostgreSQL (Data Warehouse)

Base de données OLAP utilisant un schéma en constellation.

4 Récupération des Données

L'objectif de cette section est d'automatiser la récupération et le téléchargement des données brutes depuis le système de fichier local vers le Datalake MinIO.

```

1 // Lecture des fichiers Parquet locaux
2 val df = spark.read.parquet("/data/raw/*.parquet")
3
4 // Upload vers MinIO
5 df.write.mode("overwrite").parquet(s"s3a://${bucketName}/nyc_raw/")
6
7 // Upload du fichier de reference des zones
8 val csvLocalPath = "/data/raw/taxi_zone_lookup.csv"
9 val csvOutputPath = s"s3a://${bucketName}/taxi_zone_lookup.csv"

```

Listing 1 – Upload vers MinIO

Cette implémentation permet de télécharger à la fois les fichiers parquet des courses de taxi ainsi que le fichier *taxi_zone_lookup.csv* nécessaire aux sections suivantes.

5 Ingestion et Nettoyage

5.1 Analyse des données

Les données NYC TLC Yellow Taxi contiennent les colonnes suivantes :

Colonne	Type	Description
VendorID	Integer	Fournisseur TPEP (1=CMT, 2=VeriFone)
tpep_pickup_datetime	Timestamp	Date/heure de prise en charge
tpep_dropoff_datetime	Timestamp	Date/heure de dépose
passenger_count	Integer	Nombre de passagers
trip_distance	Double	Distance en miles
RatecodeID	Integer	Code tarifaire
store_and_fwd_flag	String	Y=stocké avant envoi
PULocationID	Integer	Zone de pickup (1-265)
DOLocationID	Integer	Zone de dropoff (1-265)
payment_type	Integer	Mode de paiement
fare_amount	Double	Tarif de base
extra	Double	Suppléments
mta_tax	Double	Taxe MTA (0.50\$)
tip_amount	Double	Pourboire
tolls_amount	Double	Péages
improvement_surcharge	Double	Taxe amélioration
congestion_surcharge	Double	Taxe congestion
airport_fee	Double	Frais aéroport
total_amount	Double	Montant total

TABLE 2 – Structure des données sources NYC TLC

Ces données représentent typiquement :

Métrique	Valeur approximative
Période couverte	2020 - présent
Nombre de courses	5-10 millions
Taille brute	~200 MB (Parquet)
Après nettoyage	~180 MB

TABLE 3 – Volumétrie des données

Après nettoyage des données, nous avons pu observer les moyennes suivantes :

Métrique	Valeur moyenne
Distance	3-5 miles
Durée	15-20 minutes
Tarif total	\$15-25
Pourboire (CB)	15-20%
Taux d'outliers	1-3%

TABLE 4 – Moyennes des données nettoyées

5.2 Nettoyage des données

Dans cette partie, on s'intéresse au nettoyage, à la validation et à la transformation de données brutes chargées précédemment. Pour cela notre pipeline se divise en 6 étapes :

Étape	Description	Critères
1	Chargement zones	265 zones TLC
2	Filtrage contractuel	Passagers 1-6, Année $\geq$ 2020, IDs valides
3	Métriques dérivées	Durée, vitesse, ratio pourboire
4	Détection remboursements	Paires course/annulation
5	Détection outliers	IQR, Z-Score, règles métier
6	Écriture partitionnée	Par date_id

TABLE 5 – Étapes du pipeline de nettoyage

Les métriques dérivées sont calculées ainsi :

Colonne	Formule
trip_duration_hours	(dropoff - pickup) / 3600
avg_speed_mph	trip_distance / trip_duration_hours
tip_ratio	tip_amount / total_amount
price_per_mile	total_amount / trip_distance
date_id	Format YYYYMMDD

TABLE 6 – Métriques dérivées

Durant cette section, on s'attarde également à détecter et étiqueter les données aberrantes (outliers). Pour cela, nous avons mis en place différents critères de détections :

```

1 val MIN_DURATION_MINUTES = 2.0
2 val MAX_SPEED_MPH = 120.0
3 val SPEED_YELLOW_ZONE = 45.0
4 val SPEED_GLOBAL = 80.0
5 val MIN_DISTANCE = 0.3
6 val MAX_LOOP_DISTANCE = 5.0
7 val MAX_PRICE_PER_MILE = 25.0
8 val MAX_DURATION_HOURS = 5.0
9 val ZSCORE_THRESHOLD = 3.0

```

Listing 2 – Seuils de detection des outliers

Ces critères de détections proviennent des différentes méthodes de calcul :

— **IQR (Inter Quartile Range)** :

$$\text{Limite basse} = Q_1 - 1,5 \times IQR \quad (1)$$

$$\text{Limite haute} = Q_3 + 1,5 \times IQR \quad (2)$$

où $IQR = Q_3 - Q_1$.

Cette méthode est appliquée sur : `trip_distance`, `total_amount`, `avg_speed_mph`, `trip_duration_hours`.

— **Z-Score** :

$$Z = \frac{|x - \mu|}{\sigma} \quad (3)$$

Un trajet est considéré comme un outlier si $Z > 3$.

Des *flags* de qualité sont ensuite ajoutés à chaque entrée de données :

- `is_outlier` : Booléen indiquant une anomalie
- `outlier_reason` : "High Speed", "Price/Mile > \$25", "Duration > 5h" etc...
- `data_quality_score` : 0 (outlier), 75 (acceptable), 100 (parfait)

6 Création du Data Warehouse

6.1 Choix du schéma

L'objectif de cette partie est de construire un entrepôt de données OLAP avec un schéma en constellation. Le choix d'un **schéma en constellation** (aussi appelé *Galaxy Schema*) pour le Data Warehouse repose sur une analyse approfondie des besoins métier et des caractéristiques des données.

TABLE 7 – Comparaison des schémas dimensionnels

Schéma	Avantages	Inconvénients	Cas d'usage
Étoile	Simplicité, requêtes rapides, facile à comprendre	Redondance des données, une seule table de faits	Analyses simples, mono-processus
Flocon de neige	Normalisation, moins de redondance	Jointures complexes, performances dégradées	Espace disque limité
Constellation	Multiple tables de faits, dimensions partagées, flexible	Complexité accrue, maintenance plus lourde	Analyses multi-processus

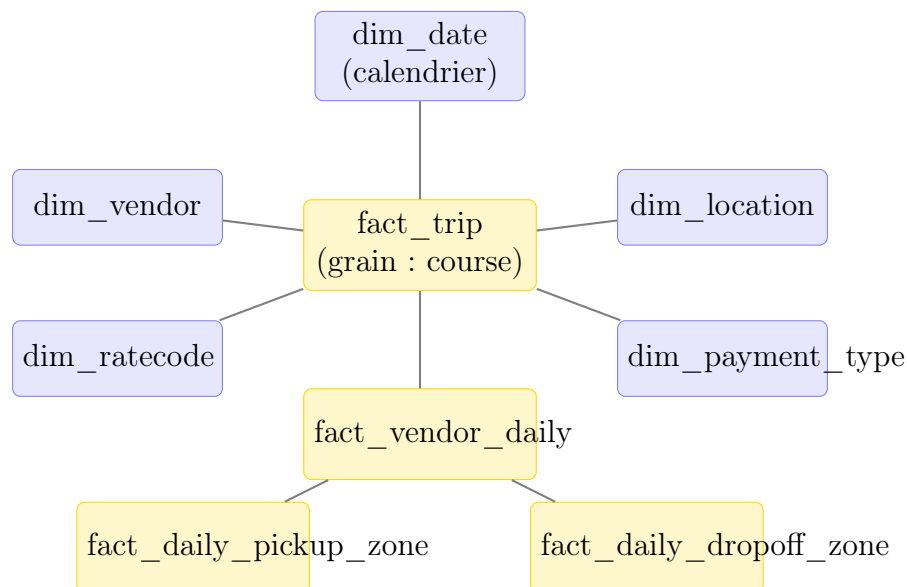


FIGURE 2 – Schéma en constellation du Data Warehouse

fact_trip	
trip_id	bigserial
date_id	int
vendor_id	int
ratecode_id	int?
payment_type_id	int?
pickup_location_id	int?
dropoff_location_id	int?
tpep_pickup_date...	timestamp
tpep_dropoff_dat...	timestamp
passenger_count	int
store_and_fwd_flag	char(1)?
trip_distance	double?
fare_amount	double?
extra	double?
mta_tax	double?
tip_amount	double?
tolls_amount	double?
improvement_surcha...	double?
congestion_surcharge	double?
airport_fee	double?
cbd_congestion_fee	double?
total_amount	double?
trip_duration_minutes	double?
avg_speed_mph	double?
tip_ratio	double?
is_outlier	boolean?
outlier_reason	varchar(255)?

(a) fact_trip

fact_vendor_daily	
date_id	int
vendor_id	int
trips_count	int?
total_fare_amount	double?
total_tip_amount	double?
total_total_amount	double?
total_distance	double?
avg_fare_amount	double?
avg_tip_amount	double?
avg_tip_ratio	double?

(b) fact_vendor_daily

fact_daily_pickup_zone	
date_id	int
pickup_location_id	int
trips_count	int?
total_fare_amount	double?
total_tip_amount	double?
total_total_amount	double?
total_distance	double?
avg_fare_amount	double?
avg_tip_amount	double?
avg_tip_ratio	double?

(c) fact_pickup_zone

fact_daily_dropoff_zone	
date_id	int
dropoff_location_id	int
trips_count	int?
total_fare_amount	double?
total_tip_amount	double?
total_total_amount	double?
total_distance	double?
avg_fare_amount	double?
avg_tip_amount	double?
avg_tip_ratio	double?

(d) fact_dropoff_zone

dim_vendor	
vendor_id	int
vendor_name	varchar(100)?

(a) dim_vendor

dim_ratecode	
ratecode_id	int
description	varchar(100)?

(b) dim_ratecode

dim_payment_type	
payment_type_id	int
description	varchar(50)?

(c) dim_payment_type

dim_location	
location_id	int
borough	varchar(50)?
zone	varchar(100)?
service_zone	varchar(50)?

(d) dim_location

dim_date	
date_id	int
full_date	date
year	int
month	int
day	int
day_of_week	int
day_name	varchar(20)?
month_name	varchar(20)?
is_weekend	boolean?

(e) dim_date

6.2 Explications

1. Multiplicité des analyses requises

Le projet nécessite plusieurs axes d'analyse distincts :

- Analyse détaillée par course (`fact_trip`)
- Performance des vendeurs par jour (`fact_vendor_daily`)
- Analyse géographique des pickups (`fact_daily_pickup_zone`)
- Analyse géographique des dropoffs (`fact_daily_dropoff_zone`)

2. Partage des dimensions

Les dimensions `dim_date`, `dim_location`, `dim_vendor` sont partagées entre plusieurs tables de faits, évitant ainsi :

- La duplication des données de référence
- Les incohérences lors des mises à jour
- La multiplication des jointures redondantes

3. Optimisation des performances

L'architecture en constellation permet de dissocier la charge de calcul de la charge de consultation. Les métriques complexes sont pré-calculées et matérialisées lors de la phase ETL (Spark), plutôt qu'exécutées à la volée par le Data Warehouse.

Cela garantit une latence minimale pour le tableau de bord (Exercice 4) qui n'interroge que des volumes réduits, tout en préservant les ressources de la base de données pour les analyses exploratoires sur la table de faits détaillée.

4. Granularités multiples

TABLE 8 – Granularité des tables de faits

Table	Grain	Usage
<code>fact_trip</code>	1 course	Analyse détaillée, ML, outliers
<code>fact_vendor_daily</code>	1 vendeur $\times$ 1 jour	KPIs vendeurs, tendances
<code>fact_daily_pickup_zone</code>	1 zone $\times$ 1 jour	Heatmaps, analyse spatiale
<code>fact_daily_dropoff_zone</code>	1 zone $\times$ 1 jour	Flux de destination

5. Évolutivité

Le schéma en constellation permet d'ajouter facilement de nouvelles tables de faits (ex : `fact_hourly_stats`, `fact_monthly_revenue`) sans modifier les dimensions existantes.

6. Séparation des préoccupations

- `fact_trip` : Données complètes pour le Machine Learning et l'analyse des outliers
- Tables agrégées : Données optimisées pour le dashboard et les KPIs temps réel

7. Rejet des alternatives

- **Schéma en étoile unique** : Rejeté car il imposerait des agrégations à la volée pour chaque visualisation du dashboard. Pour répondre aux axes d'analyse variés (vendeur, géographie), cela engendrerait des requêtes inutilement complexes et coûteuses en ressources système, dégradant l'expérience utilisateur.

- **Schéma en flocon de neige** : Rejeté car la normalisation excessive des dimensions (ex : normaliser `dim_location` en `borough` → `zone` → `service_zone`) aurait complexifié les requêtes sans apporter de bénéfice significatif pour un référentiel de seulement 265 zones.

7 Dashboard Analytique

L'objectif de cette section était de mettre en place un dashboard interactif pour la visualisation de données. Pour cela, le framewrok *Streamlit* a été utilisé avec *Plotly* pour la génération des graphiques et *SQLAlchemy* afin de connecter la base de données.

Les onglets suivants ont été implémentés :

1. **Vue d'ensemble** : KPIs, tendances journalières, score qualité
2. **Analyse Géographique** : Top zones pickup/dropoff, répartition par borough
3. **Analyse Temporelle** : Distribution horaire, variations hebdomadaires
4. **Vendeurs & Paiements** : Parts de marché, types de paiement
5. **Distributions** : Tranches de distance et tarif
6. **Analyse des Outliers** : Comparaison, patterns, échantillons détaillés

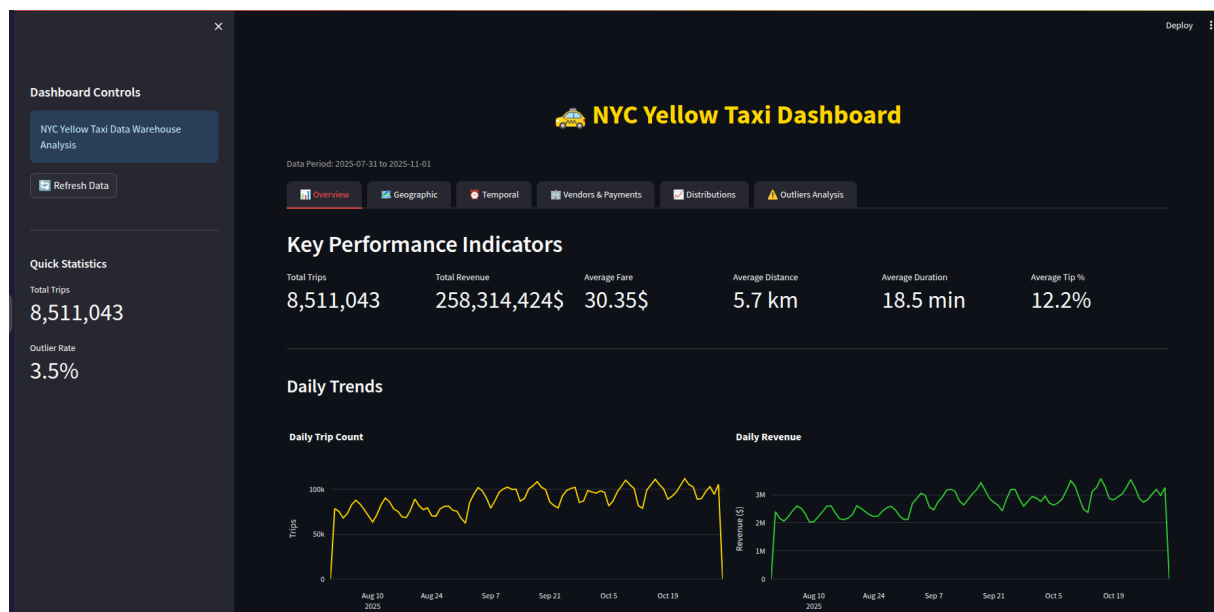


FIGURE 5 – Dashboard – Onglet Vue d'ensemble : KPIs et tendances journalières

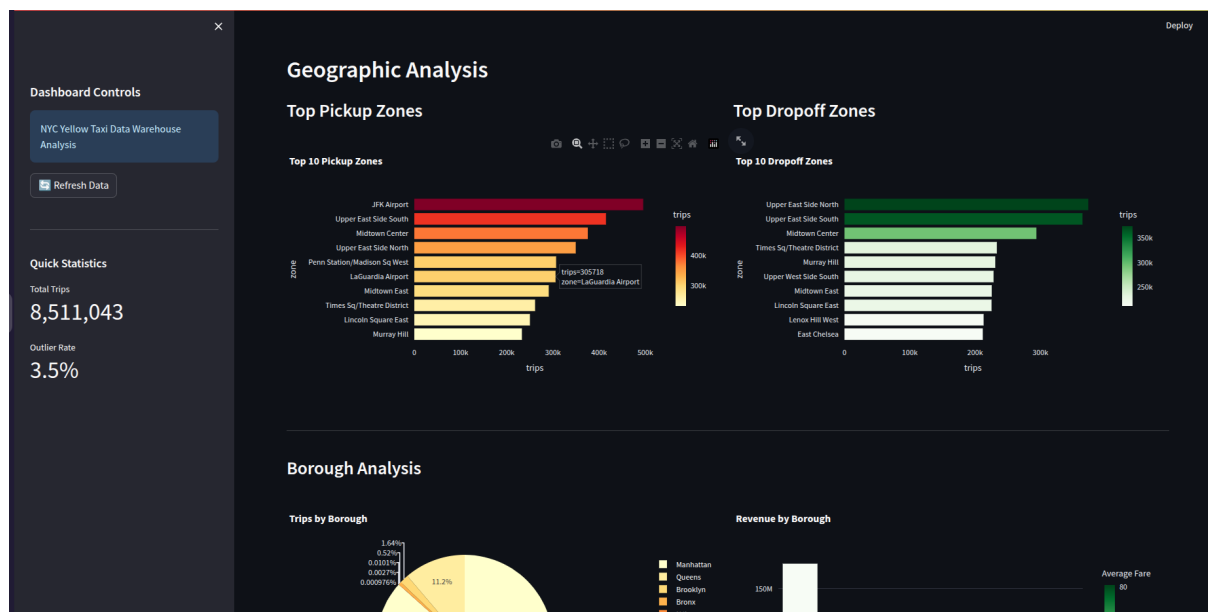


FIGURE 6 – Dashboard – Onglet Analyse Géographique : top zones et arrondissements

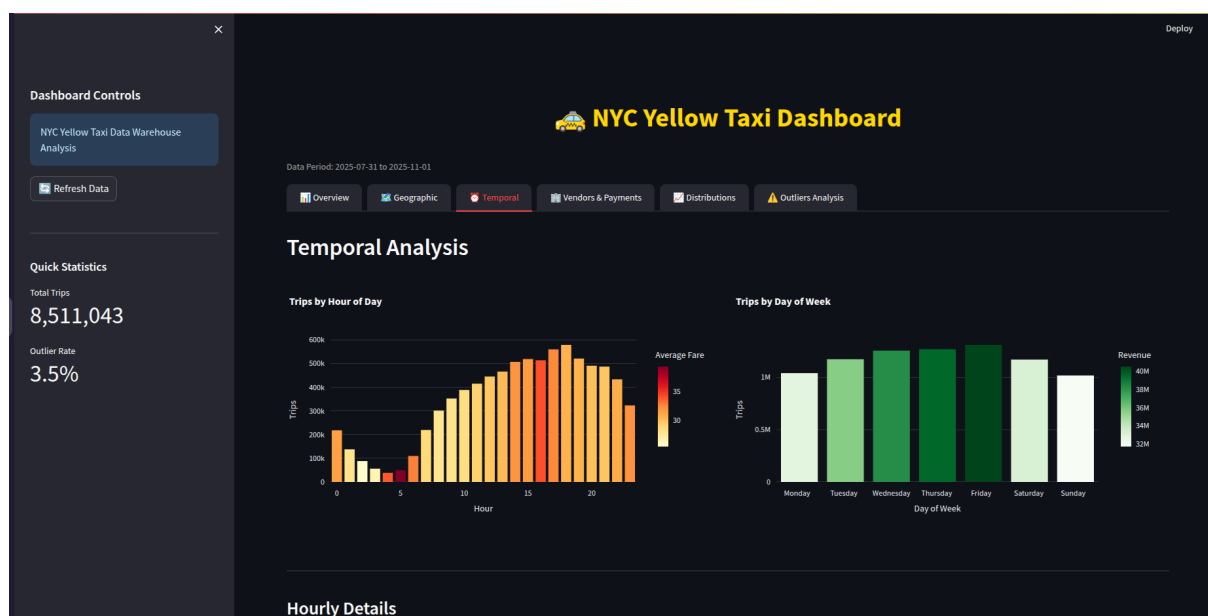


FIGURE 7 – Dashboard – Onglet Analyse Temporelle : distribution horaire et hebdomadaire

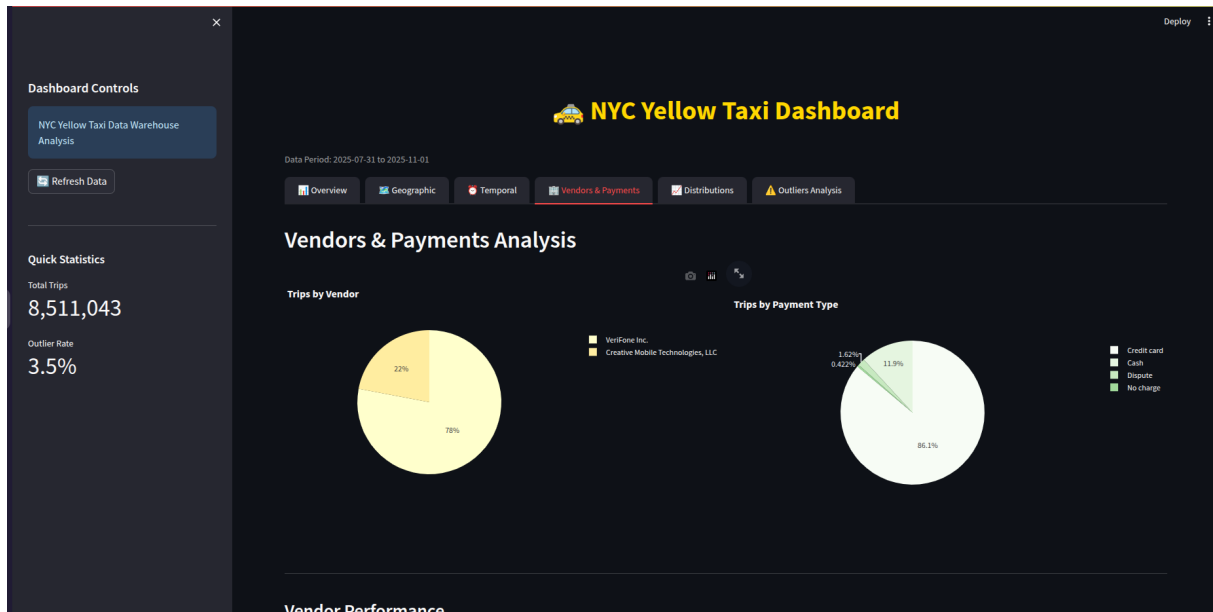


FIGURE 8 – Dashboard – Onglet Vendeurs & Paiements : parts de marché

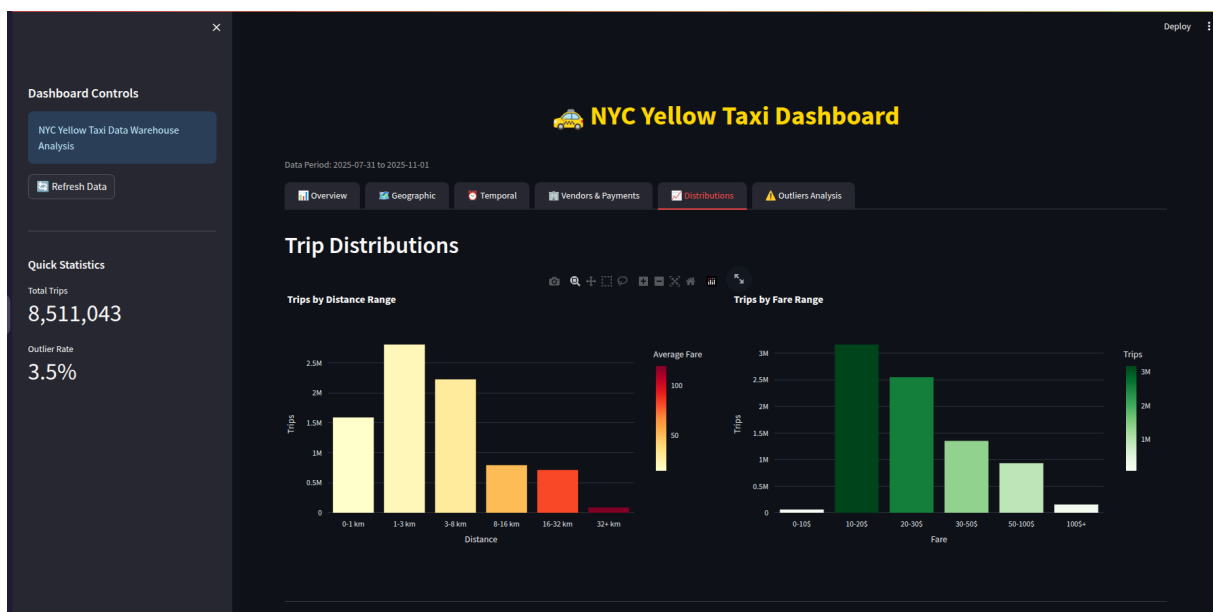


FIGURE 9 – Dashboard – Onglet Distributions : tranches de distance et tarifs

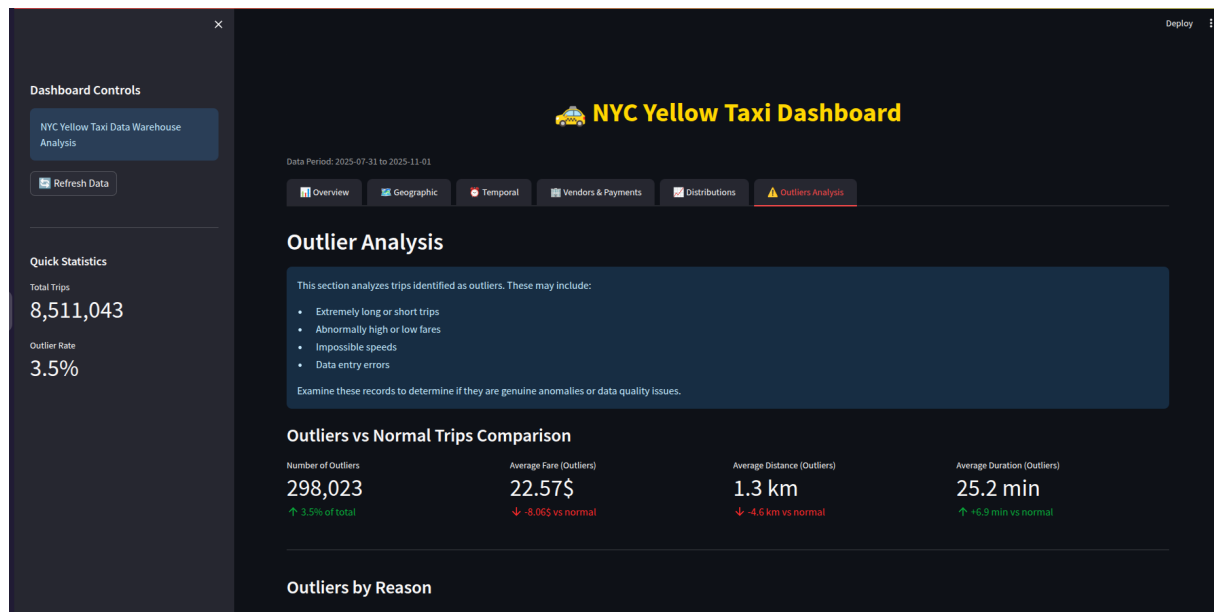


FIGURE 10 – Dashboard – Onglet Analyse des Outliers : comparaison et patterns

8 Service de prédiction ML

8.1 Modèle

Afin de prédire le prix d’une course, nous avons fait le choix d’utiliser un modèle de régression linéaire, les features suivantes ont été choisies car nous avons remarqué, lors de l’analyse des données, qu’elles étaient celles qui influençaient le plus le prix final :

Feature	Description	Type
trip_distance	Distance du trajet	Numérique
pickup_location_id	Zone de départ	Catégoriel
dropoff_location_id	Zone d’arrivée	Catégoriel
pickup_hour	Heure de prise en charge	Numérique
day_of_week	Jour de la semaine	Numérique

TABLE 9 – Features du modèle ML

Les features `pickup_hour` et `day_of_week` sont extraites de la colonne `tpep_pickup_datetime` du parquet de données.

8.2 Entraînement et évaluation

Le modèle a été entraîné sur 80% du volume de données et testé sur les 20% restants. Avant de lancer l’entraînement, des tests unitaires sont automatiquement lancés afin de s’assurer de l’intégrité et de la cohérence des données avant entraînement.

Une fois le modèle entraîné, celui-ci est évalué avec un calcul de la RMSE. Si celle-ci est inférieure à 10, le modèle est automatiquement sauvegardé. Cette vérification garantit que les modèles conservés aient une précision minimale de 10\$ autour du véritable prix.

```

Done.
Train: (1059103, 30)
Test: (264776, 30)

Preparing data...
Training model on 1059103 rows...
Evaluating model on 264776 rows...
Validation RMSE: 9.4376
Model saved to models/taxi_price_model.joblib

```

FIGURE 11 – Sortie console de l'entraînement du modèle ML (RMSE, métriques)

FIGURE 12 – Interface Streamlit du service de prédiction ML

8.3 Architecture Modulaire

```

1 ex05_ml_prediction_service/
2   src/
3       train.py           # Pipeline d'entraînement
4       inference.py       # Moteur de prediction
5       data_manager.py    # ETL pour ML
6       model_manager.py   # Persistance modele
7       app.py             # Interface Streamlit
8   models/
9       taxi_price_model.joblib
10  tests/
11      test_train.py
12      test_inference.py

```

Listing 3 – Structure du module ML

9 Orchestration Airflow

Dans cette section, on s'intéresse à l'automatisation complète du pipeline avec Apache Airflow, incluant le téléchargement automatique des dernières données disponibles depuis le site NYC TLC.

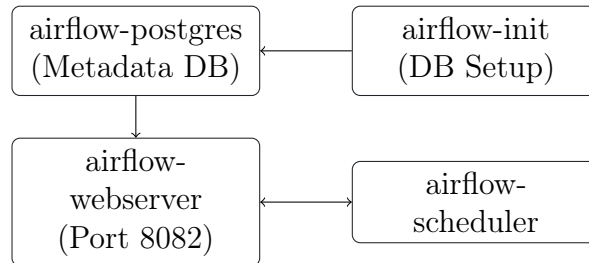


FIGURE 13 – Architecture du cluster Airflow

9.1 DAG 1 : Pipeline Complet (Manuel)

TABLE 10 – Tâches du DAG principal

Task ID	Description	Dépendances
start	Début du workflow	-
check_infrastructure	Vérifie les conteneurs	start
ex01_data_retrieval	Téléchargement données	check_infrastructure
ex02_data_ingestion	Nettoyage Spark	ex01
ex03_dwh_loading	Chargement DWH (6 sous-tâches)	ex02
ex05_ml_training	Entraînement modèle	ex03
end	Fin du workflow	ex05

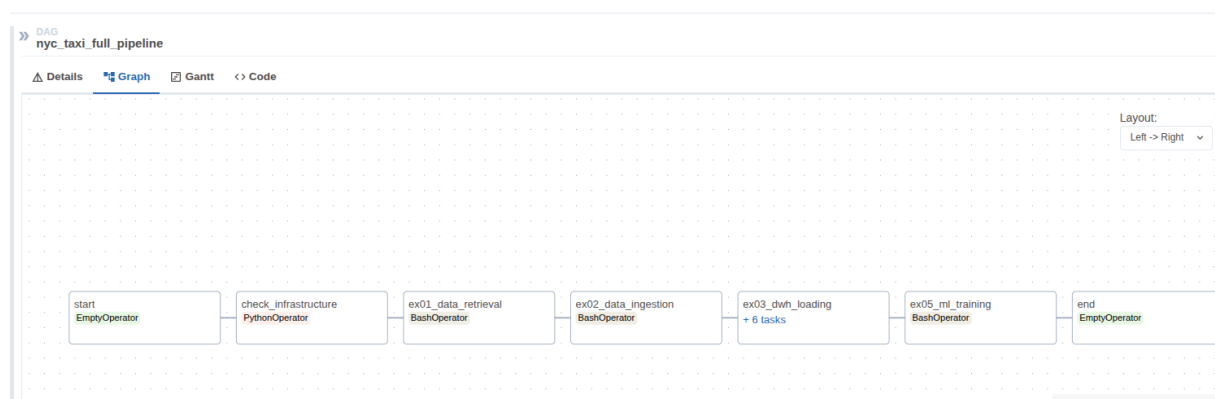
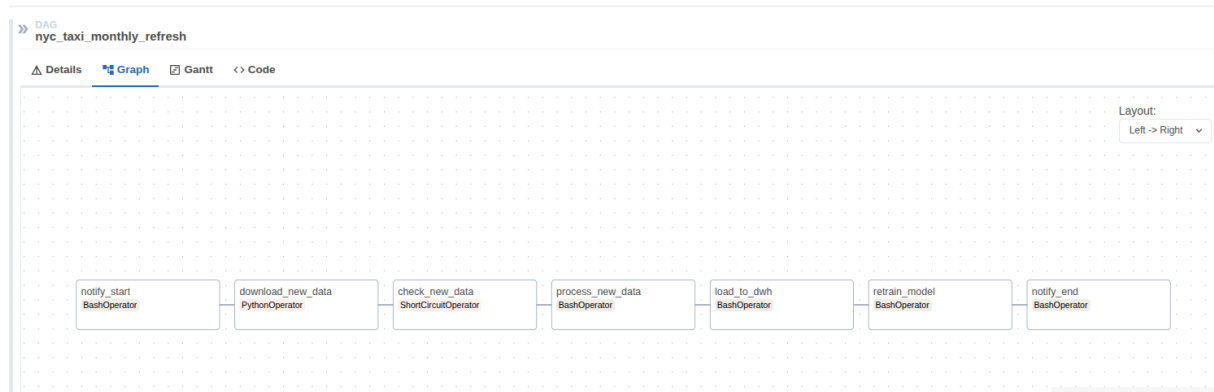


FIGURE 14 – Airflow – Graphe du DAG nyc_taxi_full_pipeline

9.2 DAG 2 : Rafraîchissement Mensuel Automatique

Le DAG mensuel télécharge **automatiquement** les dernières données disponibles depuis le site de la NYC TLC, sans intervention manuelle.

- **Schedule** : 0 2 1 * * (1er du mois à 2h00)
- **Source** : <https://d37ci6vzurychx.cloudfront.net/trip-data/>
- **Retry** : 2 tentatives, délai 2 minutes

FIGURE 15 – Airflow – Graphe du DAG `nyc_taxi_monthly`

9.2.1 Fonctionnement du Téléchargement Automatique

Le script `download_nyc_data.py` implémente la logique suivante :

1. **Inventaire MinIO** : Liste les fichiers déjà présents dans `nyc_raw/` pour identifier les mois disponibles.
2. **Calcul des mois manquants** : Compare avec les 12 derniers mois (en tenant compte du délai de publication de ~ 2 mois par la TLC).
3. **Vérification de disponibilité** : Envoie des requêtes HTTP HEAD au CDN de la TLC pour chaque mois manquant.
4. **Téléchargement et upload** : Pour chaque mois disponible, télécharge le fichier Parquet et l'uploade directement dans MinIO via le client Python `minio`.
5. **Short-circuit** : Si aucune nouvelle donnée n'est disponible, le pipeline s'arrête proprement sans erreur grâce à un `ShortCircuitOperator`.

TABLE 11 – Tâches du DAG mensuel automatique

Task ID	Description	Type
<code>notify_start</code>	Notification de démarrage	BashOperator
<code>download_new_data</code>	Téléchargement TLC $\rightarrow$ MinIO	PythonOperator
<code>check_new_data</code>	Short-circuit si rien de nouveau	ShortCircuitOperator
<code>process_new_data</code>	Nettoyage Spark (Ex02)	BashOperator
<code>load_to_dwh</code>	Chargement PostgreSQL (Ex03)	BashOperator
<code>retrain_model</code>	Ré-entraînement ML (Ex05)	BashOperator
<code>notify_end</code>	Notification de fin	BashOperator

Les fichiers Parquet sont disponibles publiquement sur le CDN CloudFront de la TLC :

```

1 # Format de l'URL
2 https://d37ci6vzurychx.cloudfront.net/trip-data/
   yellow_tripdata_YYYY-MM.parquet
3
4 # Exemples
5 yellow_tripdata_2025-08.parquet      # Aout 2025
6 yellow_tripdata_2025-09.parquet      # Septembre 2025
7 yellow_tripdata_2025-10.parquet      # Octobre 2025

```

Listing 4 – Pattern d'URL des données NYC TLC

Les données sont publiées avec un délai d'environ 2 mois : le fichier de novembre 2025 devient typiquement disponible fin janvier 2026.

10 Conclusion

Le projet BigYellowData a abouti à l'implémentation d'un pipeline ETL robuste automatisant l'intégralité du cycle de vie de la donnée, tout en garantissant une qualité optimale via une détection d'outliers multicritères reposant sur l'IQR, le Z-Score et des règles métier spécifiques. Cette architecture s'appuie sur un Data Warehouse structuré en constellation pour l'analyse OLAP, alimentant à la fois un dashboard interactif doté de six vues analytiques et un modèle de Machine Learning pour la prédiction de prix, le tout étant orchestré de manière fluide par Airflow.

Sur le plan technique, la solution se distingue par sa grande scalabilité permise par l'utilisation de Spark en mode distribué, sa reproductibilité totale grâce à la conteneurisation Docker, ainsi que sa maintenabilité assurée par un code modulaire rigoureusement documenté et testé.

Enfin, bien que le système soit opérationnel, il ouvre la voie à des évolutions majeures telles que l'intégration du streaming en temps réel avec Kafka, l'exploration de modèles de type XGBoost ou réseaux de neurones, l'ajout de visualisations géospatiales, ou encore le renforcement du monitoring et de la CI/CD via Prometheus, Grafana et GitHub Actions.

11 Annexe

```

1 # Demarrer l'infrastructure
2 docker-compose up -d
3
4 # Executer le pipeline complet
5 ./setup_and_run.sh all
6
7 # Acceder aux interfaces
8 # Spark UI:      http://localhost:8081
9 # MinIO:         http://localhost:9001
10 # pgAdmin:       http://localhost:5050
11 # Dashboard:     http://localhost:8501
12 # Airflow:       http://localhost:8082

```

Listing 5 – Commandes principales

Service	Utilisateur	Mot de passe
MinIO	minioadmin	minioadmin
PostgreSQL	user_dw	password_dw
pgAdmin	admin@admin.com	admin
Airflow	admin	admin

TABLE 12 – Identifiants de connexion